PROGRAMACIÓN AVANZADA

Sesión 04

Representaciones Binarias

→ NOTAS DE CLASE

Universidad Panamericana 2026

Índice

1 Solución: el problema de Carina **2**

2 ¿Cuántos valores caben en n bits? **2**

3 Representación de negativos — Complemento a 2 **2**
3.1 La idea 3
3.2 ¿Por qué `11111111` es -1 y no -127? 3
3.3 Cómo negar un número: voltear + 1 3

4 ASCII — El texto también son números **4**
4.1 Tabla de letras mayúsculas 4
4.2 Mayúsculas vs. minúsculas 4

5 IEEE 754 — Números con punto decimal **5**
5.1 La idea: notación científica en binario 5
5.2 Formato `float` (32 bits) y `double` (64 bits) 5
5.3 ¿Por qué `0.1 + 0.2` $\neq$ `0.3` en la computadora? 5

6 Resumen **6**

Solución: el problema de Carina

El problema de la sesión anterior: Carina escribe los enteros positivos en orden pero su calculadora no tiene las teclas del **2** ni del **7**. ¿Cuál es el milésimo número que escribe?

Los dígitos disponibles son $\{0, 1, 3, 4, 5, 6, 8, 9\}$ — exactamente **ocho** símbolos. Si los renombramos como $\{0, 1, 2, 3, 4, 5, 6, 7\}$, Carina en realidad está contando en **base 8**.

Convertimos 1000_{10} a base 8 por divisiones sucesivas:

1000 ÷ 8 = 125	resto **0**
125 ÷ 8 = 15	resto **5**
15 ÷ 8 = 1	resto **7**
1 ÷ 8 = 0	resto **1**

$1000_{10} = 1750_8$. Ahora mapeamos cada dígito octal al símbolo real de Carina:

$$0 \to 0, \quad 1 \to 1, \quad 2 \to 3, \quad 3 \to 4, \quad 4 \to 5, \quad 5 \to 6, \quad 6 \to 8, \quad 7 \to 9$$

$$1750_8 \xrightarrow{\text{remap}} \mathbf{1960}$$

✔ Recuerda

Verificación: $1 \times 8^3 + 7 \times 8^2 + 5 \times 8 + 0 = 512 + 448 + 40 = 1000$. ✓

¿Cuántos valores caben en n bits?

Con n bits existen exactamente 2^n patrones posibles. Para enteros **sin signo** (solo positivos), esos patrones representan los valores 0 a $2^n - 1$.

Bits	Patrones	Rango sin signo
8	$2^8 = 256$	0 a 255
16	$2^{16} \approx 65{,}000$	0 a 65 535
32	$2^{32} \approx 4{,}000{,}000{,}000$	0 a 4 294 967 295
64	$2^{64} \approx 1.8 \times 10^{19}$	0 a ≈18 trillones

Memoriza las potencias clave: $2^8 = 256$, $2^{10} = 1024$ (aprox. 1 000), $2^{16} = 65\,536$, $2^{32} \approx 4 \times 10^9$. Son las que aparecen constantemente al declarar tipos en C++.

Representación de negativos — Complemento a 2

Para representar números negativos hay que "partir" el rango de 2^n patrones en positivos y negativos. El método que usa **toda la computación moderna** es el **complemento a 2**.

La idea

Con n bits dedicamos **1 bit** (el más significativo, el de más a la izquierda) a indicar el signo:

- **0** en el bit más significativo $\rightarrow$ número **positivo o cero**.
- **1** en el bit más significativo $\rightarrow$ número **negativo**.

Con 8 bits el reparto queda:

Patrón	Valor
00000000	0
00000001	$+1$
$\vdots$	$\vdots$
01111111	$+127$
10000000	-128
10000001	-127
$\vdots$	$\vdots$
11111110	-2
11111111	-1

✔ Recuerda

Con n bits en complemento a 2 el rango es -2^{n-1} a $+2^{n-1}-1$. Para 8 bits: -128 **a** $+127$. Hay un negativo "extra" porque el cero ocupa un patrón del lado positivo.

¿Por qué 11111111 es -1 y no -127?

La clave: en complemento a 2, un número negativo $-x$ se almacena como $2^n - x$.

$$-1 \text{ con 8 bits } = 2^8 - 1 = 256 - 1 = 255 = \texttt{11111111}_2$$

$$-128 \text{ con 8 bits } = 2^8 - 128 = 256 - 128 = 128 = \texttt{10000000}_2$$

Cómo negar un número: voltear + 1

Para obtener $-x$ a partir de x (o viceversa):

1. **Invierte** todos los bits (0→1, 1→0).
2. **Suma 1** al resultado.

6_{10}	00000110
Invertir bits	11111001
Sumar 1	11111010

$-6_{10} = \texttt{11111010}_2$.

Verificación: $-6 = 2^8 - 6 = 250 = \texttt{11111010}_2$. ✓

El rango no es simétrico: con 8 bits existe -128 pero **no** $+128$. Si intentas negar -128 aplicando el algoritmo, el resultado desborda y obtienes -128 de nuevo. Este es un comportamiento real de las CPUs y fuente de bugs históricos.

ASCII — El texto también son números

Un archivo de texto no es más que una secuencia de bytes. El estándar **ASCII** (*American Standard Code for Information Interchange*, 1963) asigna un número a cada carácter imprimible y de control.

- **Tabla de letras mayúsculas**

Letra	Dec	Hex	Letra	Dec	Hex	Letra	Dec	Hex
A	65	41	J	74	4A	S	83	53
B	66	42	K	75	4B	T	84	54
C	67	43	L	76	4C	U	85	55
D	68	44	M	77	4D	V	86	56
E	69	45	N	78	4E	W	87	57
F	70	46	O	79	4F	X	88	58
G	71	47	P	80	50	Y	89	59
H	72	48	Q	81	51	Z	90	5A
I	73	49	R	82	52			

Letra	Binario	Hex	Decimal
M	`01001101`	4D	77
A	`01000001`	41	65
R	`01010010`	52	82

El texto “MAR” ocupa exactamente **3 bytes** en memoria.

- **Mayúsculas vs. minúsculas**

Observa el patrón binario de las mayúsculas y sus minúsculas:

	Binario	Hex	Decimal
`A`	`01000001`	41	65
`a`	`01100001`	61	97
`M`	`01001101`	4D	77
`m`	`01101101`	6D	109

La única diferencia es el **bit 5** (contando desde 0 por la derecha):

- Mayúsculas: bit 5 = **0** $\rightarrow$ patrones `010xxxxx` (0x4_ y 0x5_)

- Minúsculas: bit 5 = **1** → patrones `011xxxxx` (0x6_ y 0x7_)

Convertir mayúscula ↔ minúscula es simplemente **activar o desactivar el bit 5**, equivalente a sumar o restar 32.

`'A'` + 32 = `'a'`. `'Z'` + 32 = `'z'`.
Esto explica por qué en C++ puedes convertir entre casos con una suma: `c + 32` o, más elegante, `c ^ 0x20` (XOR con el bit 5).

El ASCII original usa 7 bits (128 caracteres). Para incluir acentos, chino, árabe y el resto de los sistemas de escritura del mundo existe **Unicode**, que hoy define más de 149 000 caracteres. La codificación más común en la web y en sistemas modernos es **UTF-8**, compatible con ASCII para los primeros 128 valores.
Referencia: `https://unicode.org`

IEEE 754 — Números con punto decimal

Los enteros son fáciles de representar en binario, pero ¿cómo representar 3.14 o 0.001? El estándar **IEEE 754** (1985) define el formato que usan prácticamente todos los procesadores del mundo.

La idea: notación científica en binario

Igual que en notación científica decimal escribimos 3.14×10^0 o 6.022×10^{23}, en binario escribimos

$$\pm\, 1.\underbrace{b_1 b_2 \cdots b_{22}}_{\text{mantisa}} \times 2^e$$

La parte entera siempre es 1 (en binario normalizado), así que no se almacena — es un **bit implícito gratis**.

Formato `float` (32 bits) y `double` (64 bits)

Tipo	Bits totales	Signo	Exponente	Mantisa
`float`	32	1	8	23
`double`	64	1	11	52

- **Signo**: 0 = positivo, 1 = negativo.
- **Exponente**: almacenado con un *bias* (127 para `float`, 1023 para `double`) para evitar negativos.
- **Mantisa**: los bits después del punto del $1.xxx\ldots$

¿Qué fracciones tienen representación exacta?

En base 10, una fracción $\frac{1}{n}$ tiene representación decimal **finita** si y solo si n no tiene factores primos distintos de **2** y **5** (los factores de 10).

Fracción	Decimal	¿Exacta?
$\frac{1}{2}$	0.5	Sí — 2 divide a 10
$\frac{1}{4}$	0.25	Sí — $4 = 2^2$
$\frac{1}{5}$	0.2	Sí — 5 divide a 10
$\frac{1}{3}$	$0.\overline{3}$	No — 3 no divide a 10
$\frac{1}{6}$	$0.1\overline{6}$	No — tiene factor 3
$\frac{1}{7}$	$0.\overline{142857}$	No — 7 no divide a 10

En base 2, el único factor primo es el **2**. Por eso solo las fracciones de la forma $\frac{1}{2^k}$ son exactas; todo lo demás es periódico.

Fracción	Binario	¿Exacta en base 2?
$\frac{1}{2}$	0.1_2	Sí
$\frac{1}{4}$	0.01_2	Sí
$\frac{1}{8}$	0.001_2	Sí
$\frac{1}{3}$	$0.\overline{01}_2$	No — patrón 01 se repite
$\frac{1}{5}$	$0.\overline{0011}_2$	No — patrón 0011 se repite
$\frac{1}{10}$	$0.0\overline{0011}_2$	No ($\frac{1}{5} \div 2$)

✔ Recuerda

La representación periódica no es un defecto de la computadora: es una propiedad matemática. $\frac{1}{3}$ es igual de "infinita" en decimal como en binario. La diferencia es que en base 10 lo vemos con $\frac{1}{3}$ pero no con $\frac{1}{10}$, y en base 2 pasa lo contrario.

• Cómo convertir la parte fraccionaria a binario

El algoritmo es análogo a las divisiones sucesivas para la parte entera, pero en sentido inverso: **multiplica por 2 repetidamente**.

1. Multiplica la parte fraccionaria por 2.
2. Si el resultado es ≥ 1: escribe un **1** y conserva solo la parte fraccionaria (resta 1).
3. Si el resultado es < 1: escribe un **0**.
4. Repite con la parte fraccionaria restante. Si llegas a 0, terminaste. Si el estado se repite, el número es **periódico**.

$$0.25 \times 2 = 0.50 \quad \rightarrow \text{bit } \mathbf{0}$$
$$0.50 \times 2 = 1.00 \quad \rightarrow \text{bit } \mathbf{1}\text{, resto } 0.00$$

$\frac{1}{4} = 0.01_2$ ✓

$$
\begin{aligned}
0.\overline{3} \times 2 &= 0.\overline{6} && \to \text{bit } \mathbf{0} \\
0.\overline{6} \times 2 &= 1.\overline{3} && \to \text{bit } \mathbf{1}\text{, resto } 0.\overline{3} \\
0.\overline{3} \times 2 &= 0.\overline{6} && \to \text{bit } \mathbf{0} \quad \text{(¡mismo estado inicial!)} \\
\vdots \times 2 &= \vdots && \vdots
\end{aligned}
$$

El estado se repite desde el primer paso: $\frac{1}{3} = 0.\overline{01}_2$

$$\frac{1}{3} = 0.010101\ldots_2$$

$$
\begin{aligned}
0.1 \times 2 &= 0.2 && \to \text{bit } \mathbf{0} \\
0.2 \times 2 &= 0.4 && \to \text{bit } \mathbf{0} \\
0.4 \times 2 &= 0.8 && \to \text{bit } \mathbf{0} \\
0.8 \times 2 &= 1.6 && \to \text{bit } \mathbf{1}\text{, resto } 0.6 \\
0.6 \times 2 &= 1.2 && \to \text{bit } \mathbf{1}\text{, resto } 0.2 \\
0.2 \times 2 &= 0.4 && \to \text{bit } \mathbf{0} \quad \text{(mismo estado que línea 2)} \\
\vdots \times 2 &= \vdots && \vdots
\end{aligned}
$$

$0.1_{10} = 0.0\overline{0011}_2$

- **¿Por qué `0.1 + 0.2` $\neq$ `0.3` en la computadora?**

Porque $0.1_{10} = 0.0\overline{0011}_2$ es un número periódico en binario. Al almacenarlo en un `float` (23 bits de mantisa) o `double` (52 bits), la secuencia infinita se **trunca**, introduciendo un error pequeño pero no nulo. Al sumar dos de esos errores el resultado no coincide exactamente con la representación truncada de 0.3.

Nunca compares flotantes con == en C++. Usa una tolerancia: `|a - b| < 1e-9`. Ignorar esto es una fuente frecuente de bugs difíciles de rastrear.

El estándar reserva patrones para casos extremos:

- **Infinito** ($\pm\infty$): exponente todo 1s, mantisa = 0. Resultado de dividir entre cero.
- **NaN** (*Not a Number*): exponente todo 1s, mantisa $\neq$ 0. Resultado de $0/0$, $\sqrt{-1}$, etc.
- **Cero**: todo 0s (con signo, por eso existe -0.0).

Resumen

✔ Recuerda

- Con n bits hay 2^n patrones; sin signo cubren 0 a $2^n - 1$.
- **Complemento a 2**: rango -2^{n-1} a $+2^{n-1} - 1$. Para negar: invertir todos los bits y sumar 1.
- `11111111` (8 bits) $= -1$; `10000000` (8 bits) $= -128$.
- **ASCII**: cada carácter es un byte. Mayúsculas = `0x41`–`0x5A`; minúsculas = mayúscula $+32$.
- $\frac{1}{n}$ es exacta en base b solo si n no tiene factores primos ajenos a b. En base 2: solo $\frac{1}{2^k}$ es exacta.
- Para convertir una parte fraccionaria a binario: multiplica por 2, anota el dígito entero, repite con el resto.
- **IEEE 754**: flotantes = signo + exponente + mantisa. Los errores de redondeo son normales; nunca compares con ==.